



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

PHƯƠNG PHÁP PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG VỚI UML

BỘ MÔN CÔNG NGHỆ PHẦN MỀM



Giới thiệu nội dung học phần

Về kiến thức

- Nắm vững quy trình ICONIX
- Hiểu chuyển đổi yêu cầu → thiết kế
- Vận dụng UML trong phân tích hệ thống

Về kỹ năng

- Phân tích, mô hình hóa yêu cầu
- Thiết kế hệ thống hướng đối tượng
- Làm việc nhóm và kiểm duyệt thiết kế



Giới thiệu nội dung học phần

Bài 1: Giới thiệu quy trình ICONIX Process

Bài 2: Mô hình hóa miền (Domain modeling)

Bài 3: Mô hình hóa Use Case (Use Case Modeling)

Bài 4: Requirements Review – Kiểm duyệt yêu cầu → **Kiểm tra TX1**

Bài 5: Robustness Analysis — Cầu nối từ Use Case đến Thiết kế

Bài 6: Preliminary Design Review — Kiểm duyệt thiết kế sơ bộ → **Kiểm tra TX 2**

Bài 7: Technical Architecture — Kiến trúc kỹ thuật

Bài 8: Sequence Diagrams

Bài 9: Critical Design Review (CDR) → **Kiểm tra TX3**



TẦM QUAN TRỌNG CỦA MÔN HỌC

Môn học này có quan trọng không → Có. Vì:

- Nó là tài liệu bắt buộc trong đề tài, đồ án, dự án...
- Nó là cơ sở để cho các lập trình viên viết code
- Nó là tài liệu để cho các mô hình AI hiện đại sinh mã nguồn, chương trình...



BÀI 1: GIỚI THIỆU QUY TRÌNH ICONIX

❖ Nội dung chính

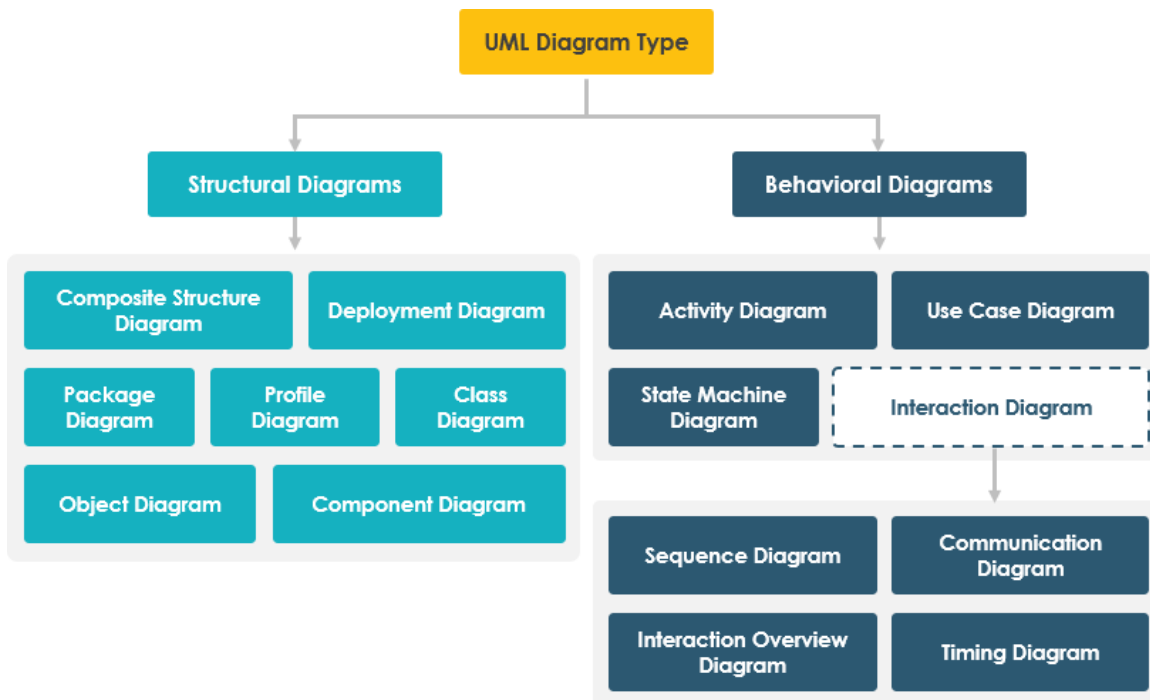
- Tổng quan quy trình ICONIX
- Kết nối yêu cầu → thiết kế → mã nguồn
- Cấu trúc 4 giai đoạn – 4 mốc kiểm duyệt
- Vai trò nền tảng của giai đoạn yêu cầu & mô hình hóa

❖ Mục tiêu

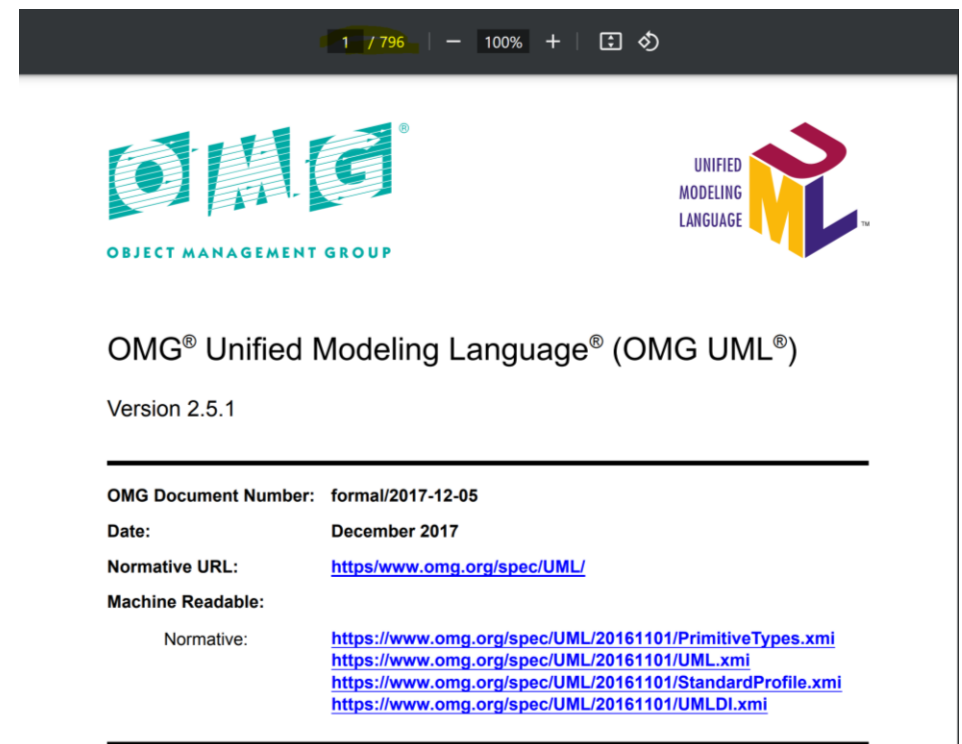
- Hiểu bản chất và vai trò của ICONIX
- Nắm 4 giai đoạn & 4 mốc kiểm duyệt
- Biết tư duy chuyển từ use case sang thiết kế và mã nguồn

TẠI SAO LẠI PHẢI HỌC & BIẾT ICONIX ?

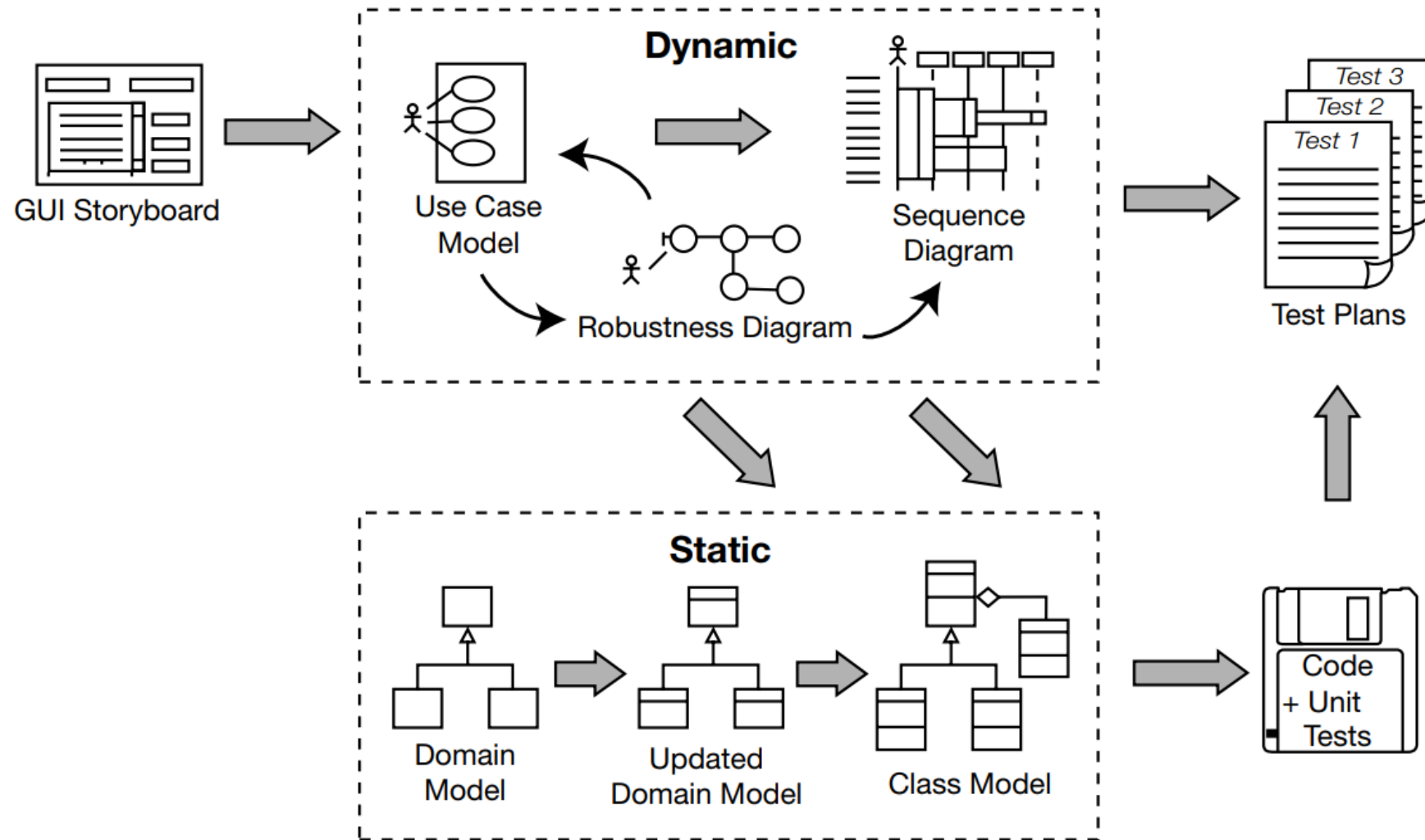
14 loại sơ đồ



~800 trang đặc tả

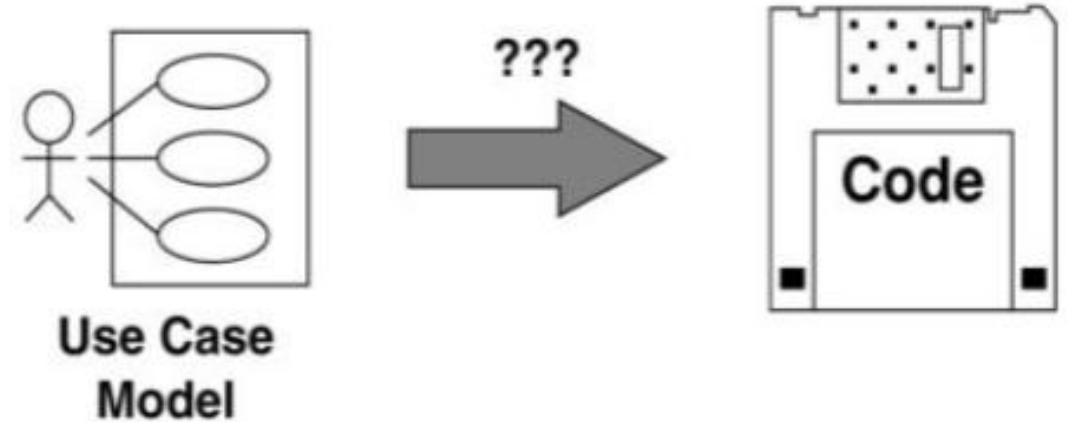


MẶT MŨI QUY TRÌNH ICONIX



VẤN ĐỀ CỤ THỂ MÀ ICONIX MUỐN GIẢI QUYẾT

- UML quá đồ sộ, nhiều lựa chọn
- Agile thường bỏ qua bước thiết kế
- “Vùng xám” giữa Use Case và Code → khó kiểm soát
- Nhiều quyết định thiết kế quan trọng bị làm vội



**Làm thế nào để từ
use case có thể tạo ra
mã nguồn?**

VÌ SAO CẦN MỘT “COOKBOOK” ?

- Mỗi dự án, mỗi nhóm đều khác nhau
- Quy trình quá “mở” → dễ mất phương hướng
- Cần một trình tự bước rõ ràng, có thể lặp lại
- ICONIX = “công thức nấu ăn” dẫn dắt qua vùng mơ hồ
- Linh hoạt thêm nêm, nhưng khung vẫn vững

MEALS WITH CHICKEN

CHICKEN BREAST WITH VEGETABLES



Tender, juicy chicken breasts are paired with a vibrant medley of roasted bell peppers, zucchini, and onions, all seasoned to perfection. Easy to prepare and full of flavor, it's a healthy choice that doesn't compromise on taste. Ideal for a quick dinner that feels like a gourmet treat!

Serves: 2

Prep Time: 15 minutes

Cook Time: 25-30 minutes

Ingredients:

- 2 boneless, skinless chicken breasts
- 1 red bell pepper, sliced
- 1 yellow bell pepper, sliced
- 1 zucchini, sliced
- 1 red onion, sliced
- 2 tablespoons olive oil
- 2 garlic cloves, minced
- 1 teaspoon dried oregano
- 1 teaspoon dried basil
- Salt and pepper to taste
- Fresh lemon juice (optional)
- Fresh parsley, chopped (for garnish)

Instructions:

1 Prepare the Chicken:

- Preheat your oven to 400°F (200°C).
- Season the chicken breasts with salt, pepper, oregano, and basil on both sides.

2 Cook the Vegetables:

- In a large bowl, toss the sliced bell peppers, zucchini, and onion with 1 tablespoon of olive oil, minced garlic, salt, and pepper.
- Spread the vegetables evenly on a baking sheet lined with parchment paper.

Agile Development with **ICONIX** Process

People, Process, and Pragmatism

A core subset of agile UML techniques, illustrated by example.

Doug Rosenberg, Matt Stephens,
and Mark Collins-Cope

Apress

ICONIX & DOANH NGHIỆP

- **Sinh ra từ dự án thật** — tiến độ gấp, tài nguyên hạn chế, yêu cầu thay đổi liên tục
- **Đủ nhẹ** → không làm chậm tiến độ (chỉ vài sơ đồ cốt lõi, đúng thời điểm)
- **Đủ cấu trúc** → tránh hỗn loạn, có milestone rõ ràng
- **Tương thích Agile** → bổ sung chỗ Agile bỏ trống, iteration ngắn vẫn đủ vòng ICONIX
- **Thực dụng, dễ áp dụng** → không cần UML guru, tập trung vào trình tự tư duy



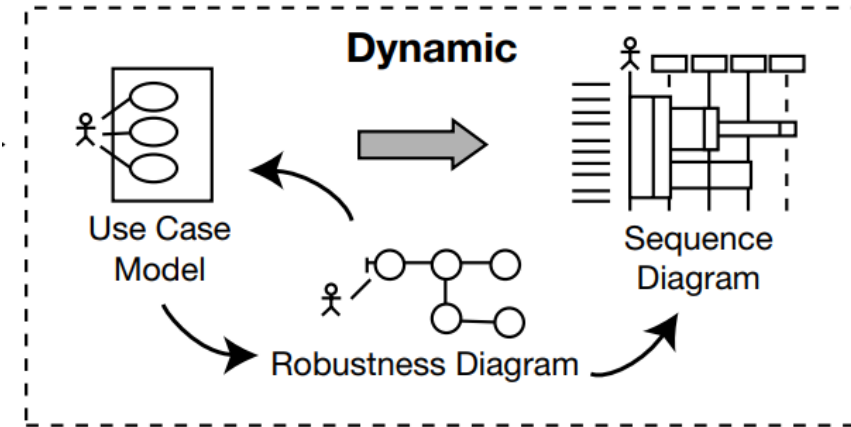
BƯỚC TRANH LỚN ICONIX

- ICONIX = “Bản đồ chỉ đường” từ **yêu cầu** → **code**
- Gồm 4 giai đoạn chính:
 1. Xác định yêu cầu
 2. Phân tích & thiết kế sơ bộ
 3. Thiết kế chi tiết
 4. Triển khai & kiểm thử
- Có cột mốc kiểm duyệt giữa các giai đoạn
- Lặp lại theo cụm use case nhỏ → phù hợp Agile

HAI LUỒNG CÔNG VIỆC TRONG ICONIX

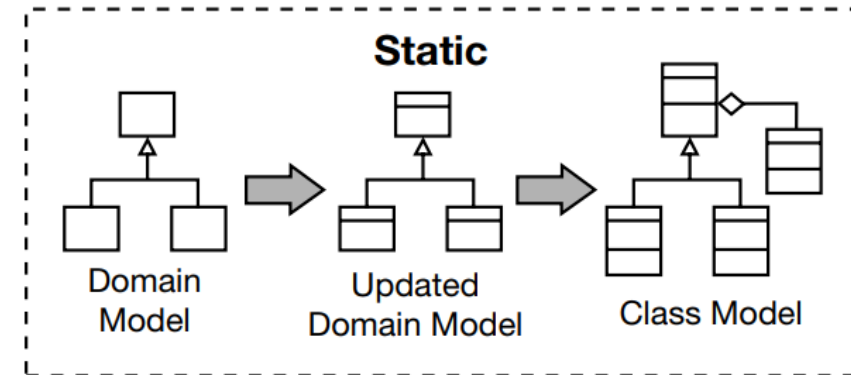
❖ Luồng động (Dynamic)

- Tập trung vào **hành vi** hệ thống
- Gồm: Use Case, Robustness, Sequence
- Trả lời: “*Hệ thống phản ứng thế nào?*”



❖ Luồng tĩnh (Static)

- Tập trung vào **cấu trúc** hệ thống
- Gồm: Domain Model, Class Diagram
- Trả lời: “*Hệ thống gồm những gì?*”





BỐN GIAI ĐOẠN CHÍNH CỦA ICONIX



Xác định yêu cầu

Làm rõ chức năng, mô hình miền sơ bộ, use case, GUI storyboard

Milestone 1: *Requirements Review*



Phân tích & Thiết kế sơ bộ

Phân tích Robustness, cập nhật domain model

Milestone 2: *Preliminary Design Review (PDR)*

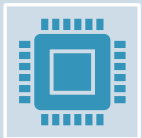


Thiết kế chi tiết

Sequence Diagram → Class Diagram

Gán hành vi vào lớp, định nghĩa phương thức

Milestone 3: *Critical Design Review (CDR)*



Triển khai

Viết code, unit test, kiểm thử tích hợp

Thiết kế tốt → code nhanh, AI hỗ trợ

Mốc

- ❶ Requirements Review
- ❷ Preliminary Design Review (PDR)
- ❸ Critical Design Review (CDR)
- ❹ Code Review & Model Update

Mục tiêu chính

Hiểu đúng yêu cầu, Use Case khớp với mong đợi khách hàng

Đảm bảo Use Case, Domain Model, Robustness nhất quán

Kiểm tra thiết kế chi tiết — Sequence & Class Diagram, không còn lỗi hổng logic

Code khớp mô hình, mô hình được cập nhật ngược lại

GIẢI ĐOẠN YÊU CẦU & MÔ HÌNH HÓA



Functional

“Hệ thống *phải làm gì*”

Danh sách chức năng

Không trả lời “*như thế nào*”

Dễ gây hiểu sai, mơ hồ



Behavioral

“Hệ thống *phản ứng thế nào*”

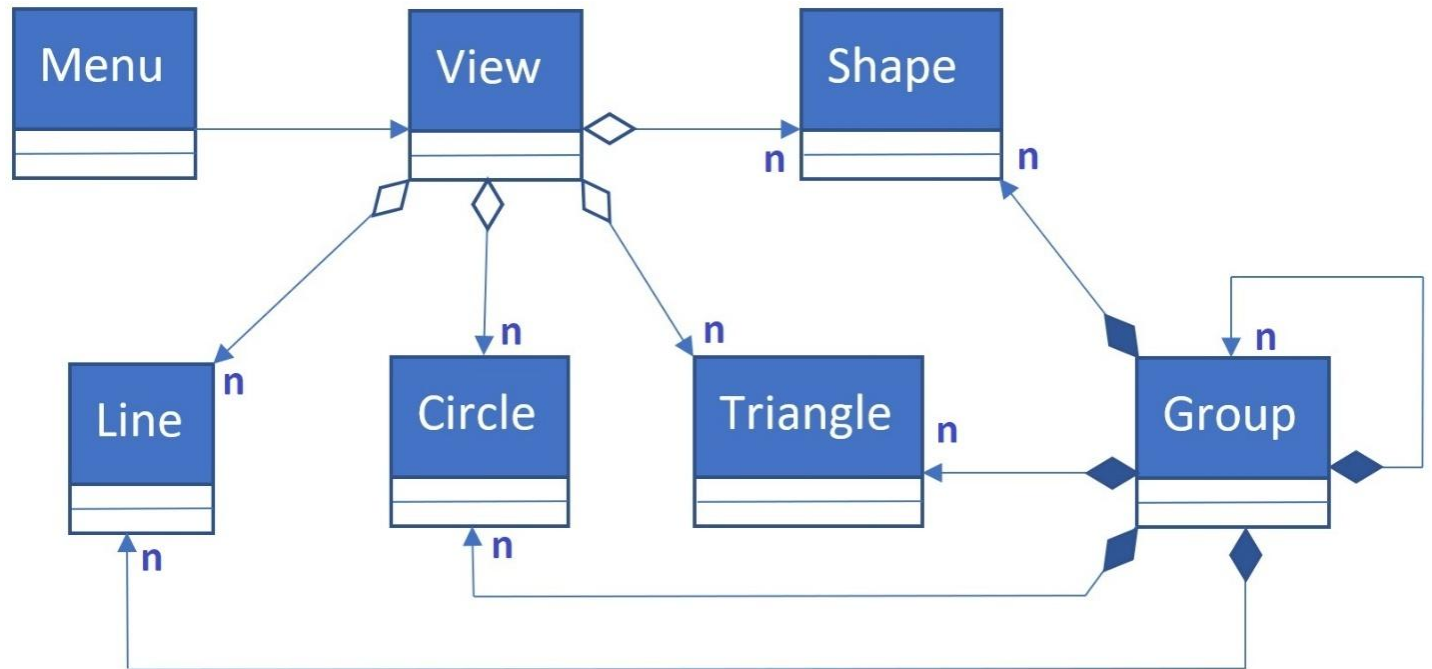
Mô tả tương tác thực tế

Thể hiện qua Use Case & GUI
Storyboard

Rõ hành vi người dùng – hệ
thống

Domain Modeling — Xây dựng ngôn ngữ chung

- Mô hình **khái niệm** → nền tảng của sơ đồ lớp (tên lớp)
- Thể hiện các **thực thể** quan trọng & **quan hệ**:
 - “is-a” (là một)
 - “has-a” (có)
- Đơn giản, dễ hiểu, không chi tiết kỹ thuật
- Phiên bản đầu tiên **luôn chưa hoàn hảo** → sẽ được tinh chỉnh dần



USE CASE MODELING

-  Ai là người tham gia? (*Actor*)
-  Mục tiêu của họ là gì?
-  Các bước họ thực hiện ra sao?
-  Hệ thống phản hồi thế nào?
-  Ngoại lệ có thể xảy ra?



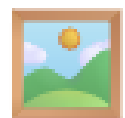
Bản vẽ tay đơn giản



Slide PowerPoint mô phỏng màn hình

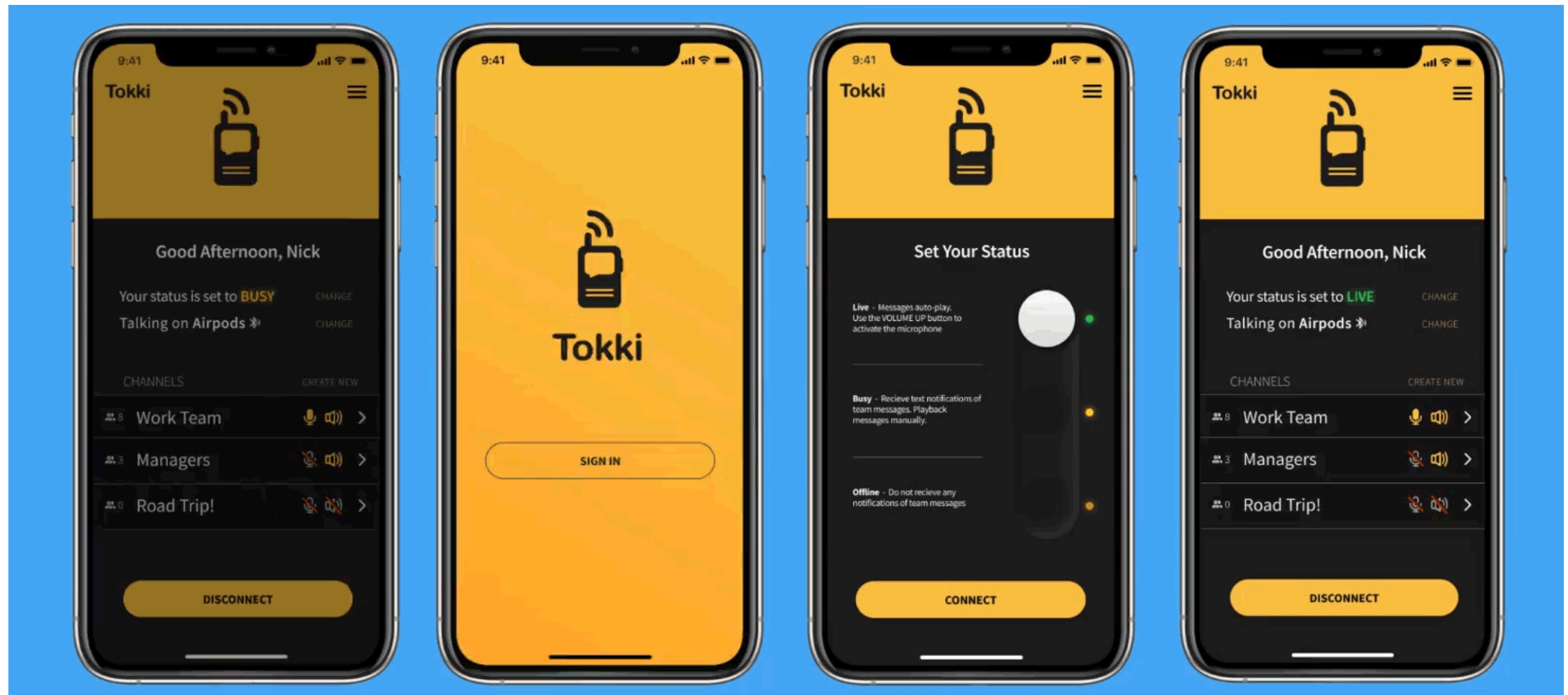


HTML prototype không backend



Mockup từ Figma / công cụ thiết kế

GUI Storyboarding



- ✓ Domain model phản ánh $\geq 80\%$ **khái niệm** quan trọng
- 📄 Use Case bao phủ **luồng chính & luồng phụ**
- 🖼️ Storyboard **khớp với Use Case**
- 👥 Rà soát đa chiều: người dùng, stakeholder, kỹ thuật

- 🛑 Không phải hợp hình thức → là **điểm kiểm soát thật sự**
- 🔍 Phát hiện sớm hiểu lầm, thiếu sót → tránh lỗi tốn kém về sau
- 🕒 Tạo nền vững chắc trước khi bước vào thiết kế



Cầu nối từ Yêu cầu → Thiết kế

- ↔ Tránh “mỗi người dịch use case một kiểu”
- 🧠 Tạo công cụ trung gian giữa Use Case ↔ Class Diagram
- 📍 Giúp thống nhất thiết kế sớm, tránh tranh cãi tùy hứng

Loại đối tượng	Vai trò chính	Ví dụ (Use Case Đặt vé xe)
■ Boundary	Giao tiếp người dùng ↔ hệ thống	Màn hình chọn tuyến, chọn ghế, thanh toán
■ Controller	Điều phối logic	DatVeController
■ Entity	Thực thể miền bài toán	Vé, Chuyến Xe, Khách Hàng

- ✓ Mọi use case đều có sơ đồ **Robustness**
- 🧠 **Entity & Boundary** trên sơ đồ ↔ domain model đầy đủ
- 🔄 **Luồng dữ liệu hợp lý**, không đứt đoạn
- 🚫 Không có khoảng trống logic
- 🏗️ Giống như “kiểm tra bản thiết kế trước khi đổ móng”
- 🔍 Giúp nhóm **tự tin bước vào thiết kế chi tiết**
- 🛑 Ngăn lỗi logic lọt sâu vào giai đoạn code

Preliminary Design Review (PDR)

Sequence Diagramming — Từ hành vi → cấu trúc

Ý tưởng chính

 Thể hiện cách các đối tượng tương tác theo thời gian

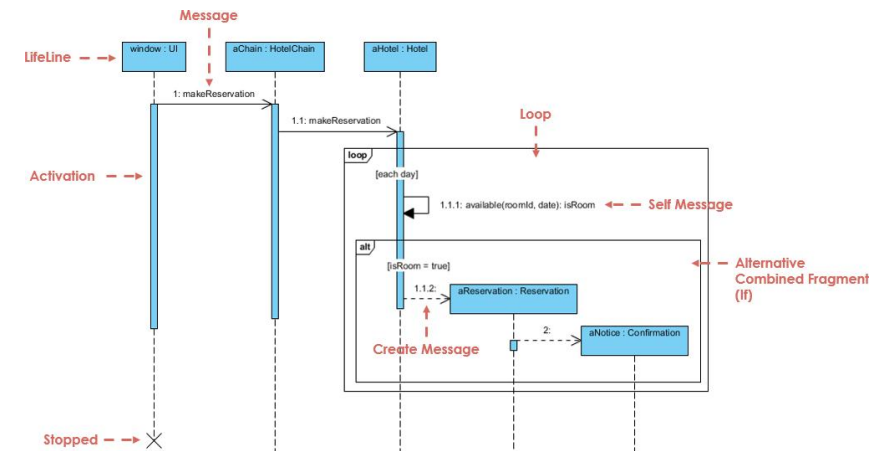
 Mỗi đối tượng = 1 lifeline (cột dọc)

 Mỗi thông điệp = 1 mũi tên

 Trình tự từ trên xuống = thứ tự thời gian

Nhiệm vụ chính trong ICONIX

- 1 Phân bổ hành vi cho lớp (behavior allocation)
- 2 Làm rõ dòng chảy logic chi tiết



KẾT LUẬN

 **ICONIX = Khung tư duy để xử lý phức tạp**

 **Hiểu đúng yêu cầu ngay từ đầu**

 **Dùng Domain Model & Use Case → tạo ngôn ngữ chung**

 **Storyboard → bám sát hành vi người dùng**

 **Robustness Diagram → chuyển yêu cầu thành cấu trúc sơ khai**

 **Sequence Diagram → hoàn thiện thiết kế hướng đối tượng**

 **Milestone Review → kiểm soát chất lượng liên tục**